

Proof notes: what each result actually says and why it's true

Hoang Nguyen

Abstract

Most theorems below sound obviously true: merging regions can't duplicate an id, a field write somewhere active can't disturb a suspended region, a closed region only opens through its front door. Intuition gives the headline, not the proof: each either hides an assumption that needs deriving from the model's invariants, or is wrong until qualified in a way you'd only notice by trying to break it. This is for someone who trusts the model (`writeup/main.tex`, `lean/gc/README.md`) but hasn't seen how these results actually got proved.

Each result: the claim, what intuition covers, where the real work is, how the proof gets there. A lemma catalog and known pitfalls follow at the end, for lookup once you know which proof you want.

Scope: `Gc/Model/` and `Gc/Reachability/Reachable/`. `Gc/Reachability/Referencable/` is excluded (deprecated, superseded by `Reachable/`; see `CLAUDE.md`).

Contents

1	Preliminaries	2
1.1	The model	2
1.2	The ten invariants of <i>ValidConfig</i>	2
1.3	<i>RuntimeConfig.start</i> satisfies <i>ValidConfig</i>	2
1.4	The preservation proof pattern	3
2	How the proofs actually work	3
2.1	<code>merge</code> keeps the configuration valid	3
2.2	Reachability, recapped	4
2.3	Reaching backward in time	4
2.4	What “Closed” really buys you	4
2.5	A field write in the active frame	5
2.6	Deleting a region reference	5
3	Reference: everything else	6
3.1	The other operations	6
3.2	<i>FRALF</i> preservation: three shapes	6
3.3	Reusable lemma catalog	7
3.4	Pitfalls	7
3.5	What's deliberately not done	8

1 Preliminaries

1.1 The model

Configuration $\langle S, H \rangle$ (stack, heap) and the nine operations' rules: full definitions in `writeup/main.tex`'s "Model"/"Operational Semantics".¹ Notation: $\text{Rld}(rid)/\text{Old}(oid)$ for references, $a.loc(\langle S, H \rangle)$ for where a reference resolves ($\text{Stk}(fid)$, $\text{Rgn}(rid)$, undefined), $a.objAt(\langle S, H \rangle)$ for the object an Old names.

1.2 The ten invariants of *ValidConfig*

`main.tex` defines all ten and bundles them into *ValidConfig*. Not restated here, just what each forbids:

Invariant	What it rules out
<i>L1</i>	the same object id allocated twice
<i>L2</i>	a frame executing inside a <i>Closed</i> region
<i>H1</i>	a region whose bridge object isn't in its own <i>objMap</i>
<i>H2</i>	two live $\text{Rld}(rid)$ references to the same region
<i>H3</i>	a region's contents pointing outside that region
<i>S1</i>	two stack frames sharing a region
<i>S2</i>	a frame referencing a stack object owned by a later frame
<i>S3</i>	a frame referencing a region owned only by a later frame
<i>HS1</i>	a dangling Old
<i>HS2</i>	a dangling Rld

Remark 1.1. *HS1/HS2* don't follow from the rest: *H2/H3/S2/S3* all presuppose a reference that already resolves somewhere, none says anything about one that resolves to nothing. Without them a stale reference could sit inert and later collide, uncaught, with a freshly allocated id. (Surfaced while proving *S2/S3* for the three `make*` operations, whose `freshObjectId/freshRegionId` only look at ids used as keys.)

Four corollaries (`Validity.lean`), cited constantly in both layers:

- `oid_in_stack_implies_not_in_heap / oid_in_heap_implies_not_in_stack`: from *L1*, an id found by one search isn't found by the other.
- `oid_loc_rgn_iff_in_heap, oid_loc_stk_iff_in_stack, rid_loc_rgn_iff_in_heap`: rewrite $\text{Old}(oid).loc(\langle S, H \rangle) = \text{Rgn}(rid)$ into $\exists region. H(rid) = region \wedge oid \in region.objMap$. Every later file reasons about where an id lives via one of these, not the raw definition.

1.3 *RuntimeConfig.start* satisfies *ValidConfig*

One root region (Open, bridge object 0), one root frame. Each invariant unfolds to a fact about a one- or two-element list, closed by `decide/simp/List.nodup_singleton`. Exists only to anchor the induction below.

¹Compiles standalone, so section names are given in prose, not as cross-file `\refs`.

1.4 The preservation proof pattern

Each `Gc/Model/Mutation/⟨Op⟩.lean` defines the operation as an *Option RuntimeConfig* do-block, each bind a failable precondition matching `main.tex`’s rule, paired with an `⟨op⟩_cases` lemma unpacking a success into an explicit disjunction, one disjunct per rule variant. This is the operation’s real specification: a missing disjunct or wrong side condition wouldn’t error here (the case lemma just replays the do-block), only later, as an unprovable or vacuous theorem downstream. Every proof below destructs $op\ cfg = some\ cfg'$ via `⟨op⟩_cases` once, then argues from the values it produces.

`Preservation/⟨Op⟩.lean` proves ten lemmas `⟨op⟩_L1, …, ⟨op⟩_HS2`, combined into `⟨op⟩_valid : ValidConfig(cfg) → op cfg = some cfg' → ValidConfig(cfg')`. `Preservation.lean` aggregates all nine, case-split on the reified *Stmt* type, into one *AllPreserve(ValidConfig)*.

2 How the proofs actually work

Five walkthroughs, covering both layers and every distinct technique. §2.1 needs only §1.2. §2.3–2.6 need §2.2’s recap of `Gc/Reachability/Reachable/` (full definitions in `main.tex`).

2.1 merge keeps the configuration valid

Claim. `merge x` folds a Closed region `rid'` into the current Open region `rid` (combines their *objMaps*, deletes `rid'`, rebinds `x` to `rid`’s old bridge object). Result is still a *ValidConfig*: no duplicate ids (*L1*), contents still resolve into their own region (*H3*).

Intuition. Pouring one region’s objects into another creates and destroys nothing, so ids stay unique and everything still points home. Right story, but “nothing created or destroyed” is a semantic claim the proof must earn from `merge`’s precondition, not its description.

The two gaps.

- *Disjointness isn’t given.* Nothing says the two regions’ object-id sets are disjoint. If not, the union could identify two different objects (breaking *L1* the other way) or, since *AList.union* erases duplicate keys from one side, silently drop an object. The precondition only gives Open/Closed; since those are mutually exclusive and *L1* forbids an id in two heap entries, that alone forces disjointness.
- *H3 needs a fresh search.* An object from the deleted `rid'` must resolve into `rid` post-merge, but `.loc(·)` has no memory: it searches whatever heap exists now. The proof reconstructs, via disjointness again, that this search lands on `rid` alone.

The argument.

1. `merge_corollary_rid_ne_regionId`: `rid, rid'` distinct, since Open $\neq$ Closed.
2. `merge_corollary_region_unique` (*L1*): no id in two heap entries, so the regions share none.
3. `merge_corollary_union_append`: on disjoint keys, $s_1 \cup s_2$ ’s entries are just $s_1.entries ++ s_2.entries$, not *AList.union*’s general, lossier form.
4. `merge_corollary_heap_entries_perm`: post-merge heap entries are a permutation of the merged pair plus everything untouched (two nested `List.exists_of_kerase` extractions).
5. *L1* then follows from permutation-invariance of `List.count/Nodup`.
6. *H3*: `merge_corollary_loc_of_mem`, since the stack side of `.loc(·)`’s search is untouched (`merge` only rewrites the current frame’s `varMap`) and the heap side’s uniqueness pins `rid`’s merged entry as the only match.

The hard part was never the union: it was showing the two regions could never have shared an id, then rebuilding where every affected object resolves under the new heap.

2.2 Reachability, recapped

$a \rightarrow b$ (*ReachableStep*): one hop. Beyond following an object’s stored references, $\text{Rld}(rid) \rightarrow b$ needs $H(rid)$ Closed and forces $b = \text{Old}(H(rid).\text{bridgeObjectId})$. A Closed region’s Rld has exactly one outgoing step; an Open region’s has none (*L2*: a step forces Closed, contradicting an on-stack owner). A chain steps into a suspended region through this one door, never out, which is why this layer exists and why §2.4 is non-trivial.

RegionReachable($rid, \cdot$)/*FrameReachable*($fid, \cdot$): inductive closures of $\rightarrow$ from a region’s bridge object, or a frame’s roots (*FrameRoot*: a variable’s value, or its region’s bridge object). *StackReachable*: *FrameReachable* from some on-stack frame. `main.tex` restates all as “a root, plus a `Relation.ReflTransGen`($\rightarrow$) chain,” the form every proof below inducts on. `ReflTransGen` has two induction principles: constructor induction (fixes the first element, grows the last; traces forward from a known root) and `head_induction_on` (fixes the last, peels off the first; chases an escaping chain from a known start). The wrong one doesn’t error, it leaves a stuck goal.

2.3 Reaching backward in time

Claim (`region_container_confined`, `stack_container_confined`). A reference chain from frame fid back to an object inside an Open region (or stack frame) implies that container’s owner has index $\leq fid$.

Intuition. You can’t reach an earlier frame’s stuff from a later one unless something already linked the two, close to *S2/S3* already.

What’s missing. *S2/S3* bound one hop. The claim is about a chain of unknown length, an opaque `ReflTransGen` witness. Going from one hop to the whole chain needs induction re-establishing the bound at every hop, with a case split at the moment a chain crosses from region to stack.

The argument.

1. Restate as *ReflTransGen* (§2.2); the inductive relation’s own recursion runs the wrong way.
2. Induct forward from the fixed start, generalized over the chain’s end. Inside the region, `predecessor_of_region_object` (*H3*: one hop from an in-region object stays in-region or escapes to stack, never a bare Rld or another region) lets you recurse. On the stack, the one-hop bound (`stack_step_region_index_le`, *S3* at single-hop level) closes it directly.
3. The two ways a chain starts (a variable slot, a frame’s bridge object) close via *S3* plus `merge_corollary_regionId_unique_index` (same region $\Rightarrow$ same index).

Here “can’t reach backward in time” stops being a slogan: the chain is unboundedly long, so a one-hop fact needs bootstrapping into an arbitrary-length one, with a real case split at the region/stack boundary.

2.4 What “Closed” really buys you

Claim (*CR6*). Once Closed: if nothing outside holds the region’s Rld, nothing inside is stack-reachable; if something does, stack-reachability inside equals region-reachability from the bridge object.

Intuition. Closed means sealed, front door or nothing, close to the definition already.

Why it isn’t just the definition. *ReachableStep* only gives the forward direction (one successor). It says nothing about predecessors, so nothing rules out some other way in already. “Sealed” is a negative claim, and proving a negative means closing every other entrance: another region’s object holding a pointer in (ruled out by *H3*: a region’s stored references resolve back into

it, so nothing external holds a live pointer in), and a stack frame still executing there (ruled out by *L2*: an on-stack frame can't own a Closed region).

The argument.

1. `predecessor_of_closed_region_object`: §2.3's lemma, Closed version. "Escaped to stack" is now impossible (*L2*'s contrapositive, `closed_region_not_owned`); the `Rld` branch (the portal) becomes available instead of ruled out.
2. `RegionReachable_of_FrameReachable_closed` / `FrameReachable_rld_of_closed_container`: backward induction, dispatching on (1)'s two cases, still inside (recurse) or portal just used (stop; the tail already witnesses the `Rld` was reachable).
3. *CR6*: stack-reachability inside a Closed region factors uniquely through the portal.

The definition of Closed gives the one allowed way in; *CR6* proves it's the only way in, needing *H3* and *L2* to cover the two entrances the definition alone leaves open.

2.5 A field write in the active frame

Claim (*FRALF* preserved by `fieldAsgn`). An object in an earlier (possibly suspended) frame's region, already visible from every later frame, stays visible after a field write in the active frame.

Intuition. A suspended region's contents aren't touched by a write elsewhere, almost the whole truth, which is exactly the danger: it stops one level too early.

Where it needs correcting. The write creates one new edge, and nothing about "active" rules out that edge connecting two things not connected before. The claim holds because anything visible from a strictly-suspended frame is, by §2.3, already confined to a root no later than that frame, strictly before the write. So the new edge is provably out of reach of any chain that matters here.

The argument.

1. Let oid_C be the mutated object's id. By §2.3 applied to oid_C , anything reaching it comes from index $\geq fid_{active}$, the active frame's own.
2. Any chain confined below fid_{active} never touches oid_C and transports untouched, covering every suspended frame automatically.
3. The remaining case: a chain rooted at the active frame. Walk it forward (`Relation.ReflTransGen.head_induction_on`), checking each hop against the new one. A match jumps onto the write's new value, which has its own root (`resolveV_frameRoot`); the argument restarts there, back in step 2's case.

The new edge really could matter; the proof shows only one frame's chain could route through it, and even that reduces to an ordinary confined argument once traced to its own root.

2.6 Deleting a region reference

Claim (*FRALF* preserved by `merge`). Same property, for the operation that deletes region `rid'` and the reference `Rld(rid')` naming it.

Intuition. `merge` relabels a heap key, creates nothing, so reachability from an earlier frame shouldn't change.

What that leaves out. `Rld(rid')`, in the merging frame's variable `x`, is deleted and replaced by `region'.bridgeObjectId`. Any argument routed through the old reference needs re-deriving through the new one; "nothing used it" has to be shown, not asserted.

The argument.

1. `merge_ridPrime_not_step_target`: $\text{Rld}(rid')$ can't be a step's target, or $H2$'s one-reference bound breaks (x 's occurrence is already the allowed one). Pure counting, no chain analysis.
2. So it's only ever a chain's root: "used the deleted reference" is a question about the start, not a hop-by-hop trace.
3. `merge_chain_transport_down`: any chain rooted elsewhere transports unconditionally.
4. The remaining case, rooted at $\text{Rld}(rid')$: `merge_ridPrime_reflTransGen_iff` shows its only first hop is its own bridge object, so the chain is the same, one hop shorter, rooted at `region'.bridgeObjectId`, exactly x 's new value.

`merge` needs its own argument because it deletes rather than edits a reference: $H2$'s bound shows the deleted reference could only be a chain's start, turning an open-ended re-routing problem into a substitution.

3 Reference: everything else

The remaining operations, lemma catalog, and pitfalls, for lookup.

3.1 The other operations

`varAsgn x yf` Bridge var: reassigns the region's bridge object; $H1$ holds since the rule's own side condition ($yfRef.loc(\cdot) = \text{Rgn}(rid)$, $rid = frame.regionId$) forces the new bridge into the same, unchanged `objMap`. Otherwise: plain local write gated by $resolveV(x, \cdot) = \epsilon$, keeping $resolveV$ unambiguous.

`fieldAsgn xf y` The stack/region rule split shows why $H3$ needs `status = Open`: a region write needs the writer's frame in that region and the new value resolving there too. Drop either and $H3$ fails. Reachability proof: §2.5.

`swap x yf` Busiest rule syntactically, but its invariant proofs are just `fieldAsgn`'s and `varAsgn`'s run side by side.

`makeObjStack x / makeObjRegion x` Fresh empty object at $\langle S, H \rangle.freshObjectId$. $L1$: `freshObjectId_not_mem`. $H2/HS1$: vacuous. `makeObjRegion` needs `status = Open`.

`makeRegion x` New Closed region, fresh bridge object. $H2$ vacuous by freshness; Closed keeps $L2$ trivial.

`merge x` §2.1, §2.6.

`enter xf bridgeVar / exit` Push/pop, flip status. $L2/S1$: index bookkeeping. Pushed frame is empty, so $S2/S3/HS1$ add nothing. Reachability: "additive" (§3.2), `safe_reflTransGen_transport` applies directly.

3.2 FRALF preservation: three shapes

1. **Additive** (`enter`, `exit`, `makeObjStack`, `makeObjRegion`, `makeRegion`, `varAsgn`): content unchanged, `safe_reflTransGen_transport` (§3.3) moves any chain for free.
2. **Content-mutating** (`fieldAsgn`, `swap`): §2.5's escape argument, owner-index bound.
3. **Reference-deleting** (`merge`): §2.6's root-only argument, $H2$ count bound.

`StackReachableInvariant /` proves the same case analysis as an $\iff$. The two directions aren't mirror images: the new edge exists in cfg' not cfg , so escape only arises going $cfg' \rightarrow cfg$.

3.3 Reusable lemma catalog

`stackWithIndex_frame_transport_down/up_of_shape_eq` cfg, cfg' agreeing up to projection π at every stack position transports `stackWithIndex` membership, preserving `.index` and π .

`stackWithIndex_objMap_get_eq_of_last_varMap_update` Last frame's `varMap` changed $\Rightarrow$ `objMap` unaffected everywhere.

`merge_corollary_regionId_unique_index` $S1$ at the `FrameWithIndex` level: same region $\Rightarrow$ same frame $\Rightarrow$ same index.

`swap_corollary_stackWithIndex_index_inj / _find_eq` Same `.index` $\Rightarrow$ same element. The most-cited lemma in `Reachable/`: collapses differently-named-but-identical frame variables.

`predecessor_of_region_object / _of_closed_region_object` Backward confinement, one hop; §2.3/§2.4.

`SafeRef / predecessor_safe / safe_reflTransGen_transport` Safe for `rid`: in `rid` or on stack, never a bare `Rld`. Propagates backward one hop; matching step relations transport a safe-ending chain wholesale. Behind the “additive” shape.

`stack_container_confined / region_container_confined` §2.3's index bound.

`RegionReachable_of_FrameReachable_closed / FrameReachable_rid_of_closed_container` Closed-region confinement behind `CR6`.

H2-forbids-a-step-target trick (§2.6) generalizes: deleting any uniquely-referenced region reference reduces transport to a root-only case split.

3.4 Pitfalls

Pitfall 3.1 (`AList.insert` reorders on an existing key). Via `kininsert/kerase`, unlike a fresh key. Entries-level equality after an insert (§2.1) needs `List.Perm (exists_of_kerase + NodupKeys.eq_of_mk_mem)`, not “insert = append.”

Pitfall 3.2 (`Status` has no `LawfulBEq`). `beq_iff_eq` doesn't fire. Open/Closed contradictions close by case-splitting `region.status` and `decide`, not rewriting `==`.

Pitfall 3.3 (the `<op>_cases` lemma is the semantics). Read it, not the opaque `do-block`, when in doubt about a precondition. A mistake there stays invisible until a downstream theorem quietly stops being provable.

Pitfall 3.4 (Open-only tools on a Closed region). §2.3's region lemma needs `status = Open` (leans on `predecessor_of_region_object`, Open-only). For Closed, use §2.4's lemmas: the Open-only one just fails to apply, an easy mismatch when adapting to a new operation.

Pitfall 3.5 (`loc(·)` treated as opaque). Unfolded once, in `Validity.lean`, to prove §1.2's three `_iff_in_*` lemmas; every later file treats it as opaque from then on.

Pitfall 3.6 (wrong `Ref1TransGen` induction principle). Constructor induction fixes the start, grows the end; `head_induction_on` fixes the end, peels the start. Wrong choice: a stuck goal, not an error. See §2.2.

Pitfall 3.7 (an $\iff$ needs both halves independently). $cfg' \rightarrow cfg$ isn't $cfg \rightarrow cfg'$ flipped: the new edge exists only in cfg' , so escape arises in one direction only. See §3.2.

3.5 What’s deliberately not done

No `Guarantees.lean`-style top-level GC-safety theorem yet. Everything above is single-step; composing over an arbitrary-length trace (the `literal report.pdf` claim) isn’t currently pursued.